

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_6c18d403-385\agent-afd342ad1e6696d3b.jsonl`
- SHA-256: `bf82aaf6602c38a0a036fa1baf3e58573c339d992d739732e5069f5112e79fb6`
- Source modified: `2026-07-06T20:56:00+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf\_6c18d403-385`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T20:52:38.498Z)

Reviewing GitHub PR #50 of the repo at C:/Users/avidu/Projects/Telegram ? "feat(policy): add F1 pipeline primitives" (branch feat/f1-policy-engine -> main).

It is the first F1 slice of ADR 0012 (policy pipeline, just merged as #40). +280/-14, 4 files:

- src/tg_video_filter/policy.py (NEW): PolicyContext, StageResult (+ pass_/fail helpers), PolicyStage Protocol, evaluate_pipeline().
- src/tg_video_filter/db.py: NEW load_policy_config/save_policy_config (raw JSON dict over policies.config_json);
load_filter_config now uses model_validate(_policy_config_json_to_dict(...)) instead of model_validate_json;
save_filter_config now MERGES cfg.model_dump(mode='json') into the existing raw config dict (preserving unknown keys) instead of replacing the whole blob with cfg.model_dump_json().
- tests/test_db.py + tests/test_policy.py: primitives tests + a preserve-unknown-keys regression test.

A checked-out copy of the PR is at C:/Users/avidu/Projects/_wt-pr50 (read files there or via 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f1-policy-engine:<path>').

Run repros with: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr50/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script> (build schema via db._apply_schema on aiosqlite ":memory:").

BACKGROUND ? why this PR exists: ADR 0012 review found that load_filter_config/save_filter_config would ERASE future pipeline keys (topics/security_checks/deep_inspect) when a user runs /set, because save re-serialized via FilterConfig (which drops unknown keys). F1 fixes that persistence boundary. The main reviewer already verified the happy path works (pipeline keys survive a /set; flat fields update). Your job: find edge cases / bugs / regressions the happy path missed.

Ground rules:

- READ actual code, cite file:line. Run repros. Authoritative gate ALREADY RUN: ruff clean, 501 passed @ 96.63%.
- Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario per finding. Rank most-severe first.
- Severity: high (real bug/regression/data-loss), medium (edge case, latent, asymmetry), low (typing/hygiene/scope-note).

DIMENSION: config-persistence correctness (db.py load/save_policy_config, load_filter_config, save_filter_config).

Investigate edge cases the happy-path test missed:

- save_filter_config read-modify-write: is the merge order correct? cfg.model_dump(mode='json') overlays onto raw_config ?

can a flat field ever be dropped, or a pipeline key ever clobbered? What if the existing config_json has a key that is BOTH a FilterConfig field and was customized (does /set correctly win)?

- Corrupt/non-object config_json: save_filter_config catches (JSONDecodeError, ValueError) -> raw_config={} (drops unknown keys silently ? acceptable?). But load_policy_config and load_filter_config do NOT catch ? they raise on corrupt config.
- Is that a behavior change vs old load_filter_config (model_validate_json)? Does any caller break?
- load_filter_config exception-type change: old model_validate_json raised pydantic ValidationError on a non-object / malformed blob; new path raises ValueError / json.JSONDecodeError. Any caller catching the old type?
- load_policy_config default when no policy returns FilterConfig().model_dump(mode='json') ? does that dict shape match what save_policy_config/save_filter_config would later write (key set, types)? Round-trip stable?
- _policy_config_dict_to_json uses sort_keys + compact separators ? any issue with existing stored (unsorted) configs, or determinism assumptions elsewhere?

Run repros. Report findings.

2. assistant (2026-07-06T20:52:41.048Z)

I'll review PR #50's config-persistence correctness. Let me start by reading the actual code in the checked-out PR copy.

3. user (2026-07-06T20:52:42.288Z)

```

1      """Policy pipeline primitives for ADR 0012.
2
3      The policy layer is deliberately small and framework-free in F1: stages receive
4      an immutable context, return a structured result, and the evaluator stops on
5      the first failure. Later Track F phases add concrete stages.
6      """
7
8      from __future__ import annotations
9
10     from dataclasses import dataclass, field
11     from typing import Protocol
12
13     from tg_video_filter.models import ContentItem, Policy, Source
14
15
16     @dataclass(frozen=True)
17     class PolicyContext:
18         """Immutable snapshot available to a policy stage."""
19
20         content_item: ContentItem
21         source: Source
22         policy: Policy
23         fetcher: object | None = None
24         budget: object | None = None
25
26
27     @dataclass(frozen=True)
28     class StageResult:
29         """Outcome of one policy stage."""
30
31         passed: bool
32         reason: str
33         stage_name: str
34         metadata: dict[str, object] = field(default_factory=dict)
35
36     @staticmethod

```

```

37     def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
38         return StageResult(
39             passed=True,
40             reason=reason,
41             stage_name=stage_name,
42             metadata=metadata,
43         )
44
45     @staticmethod
46     def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
47         return StageResult(
48             passed=False,
49             reason=reason,
50             stage_name=stage_name,
51             metadata=metadata,
52         )
53
54
55 class PolicyStage(Protocol):
56     """A single stateless gate in the policy pipeline."""
57
58     async def evaluate(self, ctx: PolicyContext) -> StageResult:
59         """Evaluate ``ctx`` and return a stage result."""
60         ...
61
62
63     async def evaluate_pipeline(
64         ctx: PolicyContext,
65         stages: list[PolicyStage],
66     ) -> tuple[bool, list[StageResult]]:
67         """Evaluate stages in order and short-circuit on the first failure."""
68         results: list[StageResult] = []
69         for stage in stages:
70             result = await stage.evaluate(ctx)
71             results.append(result)
72             if not result.passed:
73                 return False, results
74         return True, results
75
76
77 __all__ = [
78     "PolicyContext",
79     "PolicyStage",
80     "StageResult",
81     "evaluate_pipeline",
82 ]
83

```

4. user (2026-07-06T20:52:42.443Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1389 of 1872 total (28619 tokens, cap 25000). Call Read with offset=1390 limit=1389 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```

1     """SQLite persistence layer (aiosqlite).
2
3     Schema, migrations, and repository functions. See ``docs/design.md`` §3 for
4     the legacy data model and ``docs/adr/0011-provider-neutral-domain-schema.md``
5     for the provider-neutral ``sources`` / ``content_items`` / ``policies`` /
6     ``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
7
8     The repository functions take an ``aiosqlite.Connection`` so tests can inject an
9     in-memory database (``aiosqlite.connect(":memory:")``).
10    """

```

```

11
12 from __future__ import annotations
13
14 import json
15 from collections.abc import Awaitable, Callable, Iterable
16 from datetime import UTC, datetime
17
18 import aiosqlite
19
20 from tg_video_filter.models import (
21     Account,
22     Channel,
23     ContentItem,
24     DeliveryAttempt,
25     DeliveryStatus,
26     Destination,
27     FilterConfig,
28     Policy,
29     RoutingRule,
30     Source,
31     TelegramSession,
32     User,
33     VideoRecord,
34 )
35
36 # --- DDL -----
37
38 _SCHEMA = """
39 CREATE TABLE IF NOT EXISTS accounts (
40     id            INTEGER PRIMARY KEY,
41     telegram_id   INTEGER UNIQUE,
42     provider      TEXT NOT NULL DEFAULT 'telegram',
43     display_name  TEXT,
44     created_at    TEXT NOT NULL
45 );
46
47 CREATE TABLE IF NOT EXISTS telegram_sessions (
48     account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
49     phone         TEXT,
50     session_path  TEXT NOT NULL,
51     enabled       INTEGER NOT NULL DEFAULT 1,
52     created_at    TEXT NOT NULL,
53     last_login_at TEXT
54 );
55
56 CREATE TABLE IF NOT EXISTS channels (
57     id            INTEGER PRIMARY KEY AUTOINCREMENT,
58     user_id       INTEGER NOT NULL REFERENCES accounts(id),
59     channel_id    INTEGER NOT NULL,
60     title         TEXT,
61     seen_first_at TEXT NOT NULL,
62     UNIQUE (user_id, channel_id)
63 );
64
65 CREATE TABLE IF NOT EXISTS videos (
66     id            INTEGER PRIMARY KEY AUTOINCREMENT,
67     user_id       INTEGER NOT NULL REFERENCES accounts(id),
68     channel_id    INTEGER NOT NULL,
69     message_id    INTEGER NOT NULL,
70     document_id   INTEGER,
71     duration_s    INTEGER,
72     width         INTEGER,
73     height        INTEGER,
74     size_bytes    INTEGER,
75     mime_type     TEXT,

```

```

76         matched      INTEGER NOT NULL DEFAULT 0,
77         forwarded    INTEGER NOT NULL DEFAULT 0,
78         seen_at      TEXT NOT NULL,
79         UNIQUE (user_id, channel_id, message_id)
80     );
81
82     CREATE TABLE IF NOT EXISTS filter_config (
83         user_id        INTEGER PRIMARY KEY REFERENCES accounts(id),
84         config_json    TEXT NOT NULL
85     );
86
87     -- Provider-neutral domain tables (ADR 0011, Track B). ``sources``,
88     -- ``content_items``, and ``policies`` are populated by migration 003 (and by
89     -- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
90     -- are created empty here so the schema is complete ? Track C (#28) implements
91     -- the delivery logic that populates them.
92
93     CREATE TABLE IF NOT EXISTS sources (
94         id              INTEGER PRIMARY KEY AUTOINCREMENT,
95         account_id      INTEGER NOT NULL REFERENCES accounts(id),
96         provider        TEXT NOT NULL DEFAULT 'telegram',
97         provider_source_id TEXT NOT NULL,
98         title           TEXT,
99         kind            TEXT NOT NULL DEFAULT 'channel',
100        watched         INTEGER NOT NULL DEFAULT 0,
101        seen_first_at    TEXT NOT NULL,
102        last_seen_at     TEXT,
103        UNIQUE (account_id, provider, provider_source_id)
104    );
105
106     CREATE TABLE IF NOT EXISTS content_items (
107         id              INTEGER PRIMARY KEY AUTOINCREMENT,
108         account_id      INTEGER NOT NULL REFERENCES accounts(id),
109         source_id       INTEGER NOT NULL REFERENCES sources(id),
110         provider        TEXT NOT NULL DEFAULT 'telegram',
111         provider_message_id TEXT NOT NULL,
112         content_type    TEXT NOT NULL DEFAULT 'video',
113         text            TEXT,
114         link_url        TEXT,
115         media_ref       TEXT,
116         document_id     TEXT,
117         duration_s      INTEGER,
118         width           INTEGER,
119         height          INTEGER,
120         size_bytes      INTEGER,
121         mime_type       TEXT,
122         dedup_key       TEXT NOT NULL,
123         matched         INTEGER NOT NULL DEFAULT 0,
124         seen_at         TEXT NOT NULL,
125         UNIQUE (account_id, source_id, provider_message_id),
126         UNIQUE (account_id, dedup_key)
127     );
128
129     CREATE TABLE IF NOT EXISTS policies (
130         id              INTEGER PRIMARY KEY AUTOINCREMENT,
131         account_id      INTEGER NOT NULL REFERENCES accounts(id),
132         name            TEXT NOT NULL,
133         config_json     TEXT NOT NULL,
134         is_default      INTEGER NOT NULL DEFAULT 0,
135         created_at      TEXT NOT NULL,
136         updated_at      TEXT NOT NULL,
137         UNIQUE (account_id, name)
138     );
139
140     -- ADR 0011: exactly one default policy per account (structural invariant).

```

```

141 CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default
142 ON policies(account_id) WHERE is_default = 1;
143
144 CREATE TABLE IF NOT EXISTS destinations (
145     id INTEGER PRIMARY KEY AUTOINCREMENT,
146     account_id INTEGER NOT NULL REFERENCES accounts(id),
147     provider TEXT NOT NULL DEFAULT 'telegram',
148     target TEXT NOT NULL,
149     kind TEXT NOT NULL DEFAULT 'saved',
150     title TEXT,
151     is_output INTEGER NOT NULL DEFAULT 0,
152     created_at TEXT NOT NULL,
153     UNIQUE (account_id, provider, target)
154 );
155
156 CREATE TABLE IF NOT EXISTS routing_rules (
157     id INTEGER PRIMARY KEY AUTOINCREMENT,
158     account_id INTEGER NOT NULL REFERENCES accounts(id),
159     source_id INTEGER NOT NULL REFERENCES sources(id),
160     policy_id INTEGER REFERENCES policies(id),
161     destination_id INTEGER NOT NULL REFERENCES destinations(id),
162     enabled INTEGER NOT NULL DEFAULT 1,
163     priority INTEGER NOT NULL DEFAULT 0,
164     created_at TEXT NOT NULL,
165     UNIQUE (account_id, source_id, destination_id)
166 );
167
168 CREATE TABLE IF NOT EXISTS delivery_attempts (
169     id INTEGER PRIMARY KEY AUTOINCREMENT,
170     content_item_id INTEGER NOT NULL REFERENCES content_items(id),
171     destination_id INTEGER NOT NULL REFERENCES destinations(id),
172     route_id INTEGER REFERENCES routing_rules(id),
173     status TEXT NOT NULL DEFAULT 'pending',
174     attempt_count INTEGER NOT NULL DEFAULT 0,
175     last_error TEXT,
176     created_at TEXT NOT NULL,
177     delivered_at TEXT,
178     next_retry_at TEXT,
179     UNIQUE (content_item_id, destination_id)
180 );
181 """
182
183 # Note: the ``idx_videos_user_document`` partial unique index is created by
184 # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
185 # which is present on fresh tables but only added to legacy v1 DBs by the
186 # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
187 # DBs that don't yet have the column.
188
189
190 async def init_db(db_path: str) -> None:
191     """Open the database at ``db_path`` and create/migrate the schema.
192
193     Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
194     then runs any pending migrations via the ``user_version`` ladder.
195     """
196     async with aiosqlite.connect(db_path) as conn:
197         await _apply_schema(conn)
198
199
200 async def _apply_schema(conn: aiosqlite.Connection) -> None:
201     # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
202     # and a busy timeout so concurrent access waits instead of failing immediately.
203     await conn.execute("PRAGMA journal_mode = WAL")
204     await conn.execute("PRAGMA foreign_keys = ON")
205     await conn.execute("PRAGMA busy_timeout = 5000")

```

```

206     await conn.executescript(_SCHEMA)
207     # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
208     # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
209     # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
210     # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
211     # any *empty* recreations of those legacy tables from a subsequent
212     # ``init_db`` are dropped here so they don't shadow the renamed copies.
213     if await _table_exists(conn, "channels_deprecated"):
214         await conn.execute("DROP TABLE IF EXISTS channels")
215         await conn.execute("DROP TABLE IF EXISTS videos")
216         await conn.execute("DROP TABLE IF EXISTS filter_config")
217     await _apply_migrations(conn)
218     await conn.commit()
219
220
221 # --- migrations -----
222 #
223 # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
224 # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
225 # the source of truth for *fresh* databases; the migrations below reconcile
226 # *existing* databases that were created by an older schema version.
227 #
228 # To add a new migration:
229 # 1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
230 # 2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
231 # 3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
232 #    ``init_db``, and asserts the new structure.
233
234 _MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
235
236
237 def _migration(
238     version: int,
239 ) -> Callable[
240     [Callable[[aiosqlite.Connection], Awaitable[None]]],
241     Callable[[aiosqlite.Connection], Awaitable[None]],
242 ]:
243     """Register a migration that brings the DB up to ``user_version == version``."""
244
245     def decorator(
246         fn: Callable[[aiosqlite.Connection], Awaitable[None]],
247     ) -> Callable[[aiosqlite.Connection], Awaitable[None]]:
248         _MIGRATIONS.append((version, fn))
249         return fn
250
251     return decorator
252
253
254 # --- provider-neutral dedup key (ADR 0011, Option 1A) -----
255 #
256 # Pure function: computes the ``content_items.dedup_key`` value from metadata
257 # already extracted at ingestion time (no download ? honors ADR 0003). The
258 # decision tree is ordered strongest-key first; each branch is guarded so
259 # partial metadata can never collapse to a degenerate key (e.g. ``tg:fp:0:0:0``).
260 # See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
261
262
263 def compute_dedup_key(
264     *,
265     provider: str,
266     document_id: str | None = None,
267     source_id: int | str | None = None,
268     provider_message_id: str | None = None,
269     size_bytes: int | None = None,
270     duration_s: int | None = None,

```

```

271     height: int | None = None,
272     width: int | None = None,
273 ) -> str:
274     """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.
275
276     Implements the ADR 0011 Option 1A decision tree:
277
278     - **Telegram** (``provider == "telegram"``):
279         1. If ``document_id`` is set ? ``"tg:doc:{document_id}"``.
280         2. Else if **all four** fingerprint fields (size, duration, height, width)
281            are set ? ``"tg:fp:{size}:{dur}:{h}:{w}"``.
282         3. Else ? ``"tg:msg:{source_id}:{provider_message_id}"`` (Tier-0 fallback;
283            ``source_id`` is required because Telegram ``message_id`` is only
284            unique within a channel).
285     - **Other providers** (``provider != "telegram"``): fall back to the Tier-0
286       ``{provider}:msg:{source_id}:{provider_message_id}"`` shape. Future
287       providers add their own branches (e.g. ``"wa:sha256:{blob_hash}"``) here.
288
289     Raises ``ValueError`` if the fallback branch is reached without the fields
290     it requires (``source_id`` + ``provider_message_id``) ? a degenerate
291     ``dedup_key`` is worse than a loud failure at ingestion time.
292     """
293     p = provider.lower()
294     if p == "telegram":
295         if document_id is not None and document_id != "":
296             return f"tg:doc:{document_id}"
297         # All four fingerprint fields are required to avoid a degenerate
298         # ``tg:fp:0:0:0:0`` key when some metadata is missing.
299         if None not in (size_bytes, duration_s, height, width):
300             return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
301         # Tier-0 fallback.
302         if source_id is None or provider_message_id is None:
303             raise ValueError(
304                 "Telegram dedup_key fallback requires source_id and provider_message_id"
305             )
306         return f"tg:msg:{source_id}:{provider_message_id}"
307     # Future-proof default for providers without a dedicated branch.
308     if source_id is None or provider_message_id is None:
309         raise ValueError(
310             f"dedup_key for provider={provider!r} requires source_id and"
311             f"provider_message_id"
312         )
313     return f"{p}:msg:{source_id}:{provider_message_id}"
314
315
316 async def _apply_migrations(conn: aiosqlite.Connection) -> None:
317     """Run every registered migration whose version > current ``user_version``."""
318     async with conn.execute("PRAGMA user_version") as cur:
319         row = await cur.fetchone()
320         current = int(row[0]) if row else 0
321
322     for version, fn in _MIGRATIONS:
323         if version <= current:
324             continue
325         await fn(conn)
326         await conn.execute(f"PRAGMA user_version = {version}")
327
328
329 async def _column_exists(conn: aiosqlite.Connection, table: str, column: str) -> bool:
330     """Return True if ``column`` exists on ``table`` (async, PRAGMA table_info)."""
331     async with conn.execute(f"PRAGMA table_info({table})") as cur:
332         rows = await cur.fetchall()
333         return any(row[1] == column for row in rows)

```



```

334
335
336 @migration(1)
337 async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
338     """Add ``document_id`` column + partial unique index (ADR 0006 content dedup).
339
340     Idempotent: safe on fresh DBs (column/index already present) and on v1 DBs
341     created before this migration existed.
342     """
343     if not await _column_exists(conn, "videos", "document_id"):
344         await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
345     # Create the partial unique index for both fresh and migrated DBs. Uses
346     # IF NOT EXISTS so it is safe on DBs where it was already created.
347     await conn.execute(
348         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
349         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
350     )
351
352
353 @migration(2)
354 async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
355     """Split the legacy ``users`` table into ``accounts`` + ``telegram_sessions``.
356
357     ADR 0010. Integer-preserving: ``accounts.id`` copies ``users.id`` and the
358     data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,
359     so their integer ``user_id`` values still resolve to the same identity (now
360     via ``accounts.id``). The legacy ``users`` table is renamed to
361     ``users_deprecated`` and kept for one release as a rollback safety net; a
362     follow-up migration drops it.
363
364     FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with
365     ``REFERENCES accounts(id)`` so migrated DBs end up with the *same* FK shape
366     as fresh DBs. (A naive rename of ``users`` would leave SQLite rewriting
367     those FKs to point at ``users_deprecated``, diverging from fresh installs
368     and breaking inserts for new accounts ? see PR #33 review.)
369
370     Idempotent: a no-op once the legacy ``users`` table has been renamed.
371     """
372     # If the legacy users table is gone, this migration has already run
373     # (or the DB is fresh and only has accounts). Nothing to do.
374     if not await _table_exists(conn, "users"):
375         return
376
377     # Build accounts from the legacy users table. accounts.id := users.id.
378     # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
379     # and recreated here so the INSERT can populate it cleanly.)
380     await conn.execute("DROP TABLE IF EXISTS accounts")
381     await conn.execute(
382         """
383         CREATE TABLE accounts (
384             id            INTEGER PRIMARY KEY,
385             telegram_id   INTEGER UNIQUE,
386             provider      TEXT NOT NULL DEFAULT 'telegram',
387             display_name  TEXT,
388             created_at    TEXT NOT NULL
389         )
390         """
391     )
392     await conn.execute(
393         """
394         INSERT INTO accounts (id, telegram_id, provider, created_at)
395         SELECT id, id, 'telegram', created_at FROM users
396         """
397     )
398

```

```

399 # Build telegram_sessions, one row per legacy userbot user.
400 # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
401 await conn.execute("DROP TABLE IF EXISTS telegram_sessions")
402 await conn.execute(
403     """
404     CREATE TABLE telegram_sessions (
405         account_id INTEGER PRIMARY KEY REFERENCES accounts(id),
406         phone TEXT,
407         session_path TEXT NOT NULL,
408         enabled INTEGER NOT NULL DEFAULT 1,
409         created_at TEXT NOT NULL,
410         last_login_at TEXT
411     )
412     """
413 )
414 # session_path is built from the *telegram_id* (= old users.id) to match
415 # the convention telethon_client.build_client used pre-migration. The
416 # sessions_dir prefix is prepended at read time (Settings.sessions_dir)
417 # so this migration does not need to know the runtime path.
418 await conn.execute(
419     """
420     INSERT INTO telegram_sessions (account_id, phone, session_path, enabled,
created_at)
421     SELECT id, phone, 'user_' || id || '.session', enabled, created_at
422     FROM users
423     """
424 )
425
426 # Rebuild channels / videos / filter_config so their FK constraints point
427 # at accounts(id) instead of the legacy users(id). SQLite's
428 # ALTER TABLE RENAME rewrites FK references in *other* tables to follow
429 # the rename ? so a naive rename of users ? users_deprecated would leave
430 # these tables referencing users_deprecated(id), diverging from fresh DBs
431 # (which reference accounts(id)) and breaking inserts for new accounts.
432 # The rebuild: rename the data-holding legacy table out of the way, create
433 # the new one with the correct FK, copy the data back, drop the renamed copy.
434 for table, new_ddl, copy_cols in (
435     (
436         "channels",
437         """
438         CREATE TABLE channels (
439             id INTEGER PRIMARY KEY AUTOINCREMENT,
440             user_id INTEGER NOT NULL REFERENCES accounts(id),
441             channel_id INTEGER NOT NULL,
442             title TEXT,
443             seen_first_at TEXT NOT NULL,
444             UNIQUE (user_id, channel_id)
445         )
446         """,
447         "id, user_id, channel_id, title, seen_first_at",
448     ),
449     (
450         "videos",
451         """
452         CREATE TABLE videos (
453             id INTEGER PRIMARY KEY AUTOINCREMENT,
454             user_id INTEGER NOT NULL REFERENCES accounts(id),
455             channel_id INTEGER NOT NULL,
456             message_id INTEGER NOT NULL,
457             document_id INTEGER,
458             duration_s INTEGER,
459             width INTEGER,
460             height INTEGER,
461             size_bytes INTEGER,
462             mime_type TEXT,

```

```

463         matched    INTEGER NOT NULL DEFAULT 0,
464         forwarded   INTEGER NOT NULL DEFAULT 0,
465         seen_at     TEXT NOT NULL,
466         UNIQUE (user_id, channel_id, message_id)
467     )
468     """
469     "id, user_id, channel_id, message_id, document_id, duration_s, width, "
470     "height, size_bytes, mime_type, matched, forwarded, seen_at",
471 ),
472 (
473     "filter_config",
474     """
475     CREATE TABLE filter_config (
476         user_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
477         config_json TEXT NOT NULL
478     )
479     """
480     "user_id, config_json",
481 ),
482 ):
483     await conn.execute(f"ALTER TABLE {table} RENAME TO {table}__legacy_data")
484     await conn.execute(new_ddl)
485     await conn.execute(
486         f"INSERT INTO {table} ({copy_cols}) SELECT {copy_cols} FROM
{table}__legacy_data"
487     )
488     await conn.execute(f"DROP TABLE {table}__legacy_data")
489
490     # The partial unique index for document_id dedup (created by migration 001
491     # against the legacy videos table) was dropped along with the legacy copy
492     # above. Recreate it on the new videos table.
493     await conn.execute(
494         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
495         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
496     )
497
498     # Rename the legacy users table (do NOT drop ? kept as a rollback safety
499     # net for one release per ADR 0010). The data-holding channels/videos/
500     # filter_config tables were already rebuilt above with FKs to accounts(id),
501     # so renaming users no longer retargets any live FK.
502     await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
503
504
505 @_migration(3)
506 async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
507     """Land the provider-neutral domain tables (ADR 0011, Track B).
508
509     Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
510     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
511     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
512     them), then backfills the three populated ones from the legacy
513     ``channels`` / ``videos`` / ``filter_config`` tables and renames the
514     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
515     pattern). Idempotent: a no-op once the legacy tables have been renamed.
516     """
517     # Idempotency gate: if the legacy videos table is gone, this migration has
518     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
519     # dropped by _apply_schema post-003). Nothing to backfill.
520     if not await _table_exists(conn, "videos"):
521         return
522
523     # 1a. Backfill sources from channels first (known title + watched=1). Order
524     # matters: channels-first so title/watched data wins; the videos-based
525     # backfill below fills gaps for orphan channels (review finding #1).
526     await conn.execute(

```

```

527         """
528         INSERT OR IGNORE INTO sources
529             (account_id, provider, provider_source_id, title, kind, watched,
530              seen_first_at, last_seen_at)
531         SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
532                1, seen_first_at, seen_first_at
533         FROM channels
534         """
535     )
536
537     # 1b. Backfill sources from videos for channels that have no channels row
538     #     (e.g. backfill-only channels that were never seen via live traffic).
539     #     INSERT OR IGNORE is a no-op on (account, provider, source_id) conflicts
540     #     ? the channels-based row above survives with its title + watched=1.
541     await conn.execute(
542         """
543         INSERT OR IGNORE INTO sources
544             (account_id, provider, provider_source_id, kind, watched, seen_first_at)
545         SELECT v.user_id, 'telegram', CAST(v.channel_id AS TEXT),
546                'channel', 0, MIN(v.seen_at)
547         FROM videos v
548         GROUP BY v.user_id, v.channel_id
549         """
550     )
551
552     # 2. Backfill content_items from videos in two phases (review findings #2 & #3):
553     #     Phase A ? insert every row with a guaranteed-unique tg:msg: Tier-0 key
554     #     (no silent drops, even on fingerprint collisions).
555     #     Phase B ? promote to tg:doc: for items with a document_id (one per doc-id
556     group).
557     #     Phase C ? promote remaining non-doc items to tg:fp: (one per fingerprint
558     group).
559     #     The non-promoted losers stay at tg:msg: (unique) ? no silent data loss.
560     await conn.execute(
561         """
562         INSERT INTO content_items
563             (account_id, source_id, provider, provider_message_id, content_type,
564              document_id, duration_s, width, height, size_bytes, mime_type,
565              dedup_key, matched, seen_at)
566         SELECT
567             v.user_id,
568             s.id,
569             'telegram',
570             CAST(v.message_id AS TEXT),
571             'video',
572             CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
573             v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
574             'tg:msg:' || s.id || ':' || v.message_id, -- Tier-0 (always unique)
575             v.matched,
576             v.seen_at
577         FROM videos v
578         JOIN sources s
579             ON s.account_id = v.user_id
580             AND s.provider = 'telegram'
581             AND s.provider_source_id = CAST(v.channel_id AS TEXT)
582         """
583     )
584
585     # Phase B: promote one row per (account, document_id) to tg:doc:.
586     await conn.execute(
587         """
588         UPDATE content_items SET dedup_key = 'tg:doc:' || document_id
589         WHERE document_id IS NOT NULL
590             AND dedup_key LIKE 'tg:msg:%'
591             AND id IN (

```

```

590         SELECT MIN(id) FROM content_items
591         WHERE document_id IS NOT NULL AND dedup_key LIKE 'tg:msg:%'
592         GROUP BY account_id, document_id
593     )
594     """
595 )
596
597 # Phase C: promote one non-doc row per (account, fingerprint) to tg:fp:.
598 await conn.execute(
599     """
600     UPDATE content_items SET dedup_key =
601         'tg:fp:' || size_bytes || ':' || duration_s || ':'
602         || height || ':' || width
603     WHERE document_id IS NULL
604         AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
605         AND height IS NOT NULL AND width IS NOT NULL
606         AND dedup_key LIKE 'tg:msg:%'
607         AND id IN (
608             SELECT MIN(id) FROM content_items
609             WHERE document_id IS NULL
610                 AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
611                 AND height IS NOT NULL AND width IS NOT NULL
612                 AND dedup_key LIKE 'tg:msg:%'
613             GROUP BY account_id, size_bytes, duration_s, height, width
614         )
615     """
616 )
617
618 # 3. Backfill policies from filter_config: one 'default' policy per account
619 #     (Option 2A). is_default=1 so the legacy load_filter_config shim reads
620 #     this row. UNIQUE(account_id, name) dedups idempotent re-runs.
621 await conn.execute(
622     """
623     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
624                                     created_at, updated_at)
625     SELECT user_id, 'default', config_json, 1,
626            strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
627            strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
628     FROM filter_config
629     """
630 )
631
632 # 4. Create the policies default-partial index (sig after backfill so the
633 #     UNIQUE constraint only applies once rows are populated). On fresh DBs
634 #     _SCHEMA creates it; on migrated DBs this is the first time.
635 await conn.execute(
636     "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
637     "ON policies(account_id) WHERE is_default = 1"
638 )
639
640 # 5. Assert no silent video loss: on a legacy DB, every videos row MUST
641 #     produce a content_items row. A mismatch signals fingerprint collisions
642 #     where two distinct videos shared (size,dur,h,w) ? the losers stay at
643 #     tg:msg: keys and are still present. Log a warning so the operator can
644 #     investigate; the data is preserved in *_deprecated tables.
645 async with conn.execute("SELECT COUNT(*) FROM videos") as cur:
646     vc = (await cur.fetchone())[0]
647 async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
648     cic = (await cur.fetchone())[0]
649 if vc != cic:
650     import logging
651
652     _logger = logging.getLogger(__name__)
653     _logger.warning(
654         "migration_003: %d videos rows produced %d content_items rows "

```

```

655         "(%d gap). Colliding items remain in videos_deprecated.",
656         vc,
657         cic,
658         vc - cic,
659     )
660
661     # 6. Rename (don't drop) the legacy tables ? kept for one release as a
662     #     rollback safety net (ADR 0010 pattern).
663     await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
664     await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
665     await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
666
667
668     @_migration(4)
669     async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
670         """Backfill default destinations and delivery state (ADR 0011, Track C).
671
672         Creates a default destination row for each account that has a
673         ``filter_config_deprecated.delivery_target`` value, then creates a default
674         routing rule (source ? default policy ? destination) for each source.
675         Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
676         table (one ``delivered`` attempt per previously-forwarded video to the
677         account's default destination).
678
679         Idempotent: skips accounts whose default destination already exists.
680         """
681         # Gate: if destinations already have data, this migration has run.
682         async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
683             if (await cur.fetchone())[0] > 0:
684                 return
685
686         if not await _table_exists(conn, "filter_config_deprecated"):
687             return
688
689         # 1. Backfill default destinations from filter_config_deprecated.
690         #     Parse the JSON delivery_target, defaulting to "me".
691         await conn.execute(
692             """
693             INSERT OR IGNORE INTO destinations (account_id, provider, target, kind,
694             is_output, created_at)
695             SELECT user_id, 'telegram',
696                 COALESCE(
697                     CASE WHEN json_valid(config_json)
698                         THEN json_extract(config_json, '$.delivery_target') END,
699                     'me'
700                 ),
701                 CASE
702                     WHEN COALESCE(
703                         CASE WHEN json_valid(config_json)
704                             THEN json_extract(config_json, '$.delivery_target') END,
705                         'me'
706                     ) = 'me'
707                     THEN 'saved'
708                     ELSE 'channel'
709                 END,
710                 0,
711                 strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
712             FROM filter_config_deprecated
713             """
714         )
715
716         # 2. Create a default routing rule per source (source ? default policy ? default
717         #     destination).
718         await conn.execute(
719             """

```

```

718         INSERT OR IGNORE INTO routing_rules
719             (account_id, source_id, policy_id, destination_id, enabled, priority,
created_at)
720         SELECT s.account_id, s.id, p.id, d.id, 1, 0,
721             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
722         FROM sources s
723         JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
724         JOIN destinations d ON d.account_id = s.account_id
725         WHERE d.target = (
726             SELECT COALESCE(
727                 CASE WHEN json_valid(fcd.config_json)
728                     THEN json_extract(fcd.config_json, '$.delivery_target') END,
729                 'me'
730             )
731             FROM filter_config_deprecated fcd
732             WHERE fcd.user_id = s.account_id
733             LIMIT 1
734         )
735         """
736     )
737
738     # 3. Backfill delivery_attempts from previously-forwarded videos.
739     await conn.execute(
740         """
741         INSERT OR IGNORE INTO delivery_attempts
742             (content_item_id, destination_id, route_id, status, attempt_count,
743             created_at, delivered_at)
744         SELECT ci.id, d.id, rr.id, 'delivered', 1,
745             vd.seen_at, vd.seen_at
746         FROM videos_deprecated vd
747         JOIN content_items ci
748             ON ci.account_id = vd.user_id
749             AND ci.provider = 'telegram'
750             AND ci.provider_message_id = CAST(vd.message_id AS TEXT)
751         JOIN sources sc
752             ON sc.id = ci.source_id
753             AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
754         JOIN destinations d ON d.account_id = vd.user_id
755         JOIN routing_rules rr
756             ON rr.account_id = vd.user_id
757             AND rr.source_id = sc.id
758             AND rr.destination_id = d.id
759         WHERE vd.forwarded = 1
760         """
761     )
762
763
764     async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
765         """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
766         async with conn.execute(
767             "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
768         ) as cur:
769             return await cur.fetchone() is not None
770
771
772     # --- helpers -----
773
774
775     def _now() -> datetime:
776         """Timezone-aware UTC 'now', used for policy timestamps."""
777         return datetime.now(UTC)
778
779
780     def _safe_int_cast(value: object, field_name: str) -> int:
781         """Cast a provider_source_id / provider_message_id / document_id to int.

```

```

782
783 All existing data is Telegram ? these ids are numeric strings. Non-Telegram
784 providers cannot be projected onto the legacy ``Channel`` / ``VideoRecord``
785 shapes (which carry Telegram-specific int ids); this function raises a clear
786 ``ValueError`` rather than letting a cryptic ``int()`` crash propagate
787 (ADR 0011 provider-neutrality guard ? review finding #5).
788 """
789 try:
790     return int(value) # type: ignore[arg-type]
791 except (ValueError, TypeError) as exc:
792     raise ValueError(
793         f"Cannot cast {field_name}={value!r} to int. The legacy Channel/VideoRecord
794         f"types are Telegram-specific; use ContentItem/Source directly for "
795         f"non-Telegram providers (ADR 0011)."
796     ) from exc
797
798
799 def _row_to_account(row: aiosqlite.Row) -> Account:
800     return Account(
801         id=row["id"],
802         telegram_id=row["telegram_id"],
803         provider=row["provider"],
804         display_name=row["display_name"],
805         created_at=row["created_at"], # stored as ISO string; pydantic parses
806     )
807
808
809 def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
810     return TelegramSession(
811         account_id=row["account_id"],
812         phone=row["phone"],
813         session_path=row["session_path"],
814         enabled=bool(row["enabled"]),
815         created_at=row["created_at"],
816         last_login_at=row["last_login_at"],
817     )
818
819
820 def _account_session_to_user(account: Account, session: TelegramSession | None) -> User:
821     """Project an (account, optional session) pair onto the legacy ``User`` view.
822
823     Used by the deprecated ``upsert_user``/``get_user``/``list_enabled_users``
824     shims so pre-ADR-0010 callers and tests keep working. A bot-only account
825     (no session) maps to a ``User`` with ``enabled=False`` so the legacy
826     "list enabled users" semantics exclude it ? matching the structural rule
827     that bot-only users are not runnable Telethon clients.
828     """
829     if session is None:
830         return User(
831             id=account.telegram_id if account.telegram_id is not None else account.id,
832             phone=None,
833             enabled=False,
834             created_at=account.created_at,
835         )
836     return User(
837         id=account.telegram_id if account.telegram_id is not None else account.id,
838         phone=session.phone,
839         enabled=session.enabled,
840         created_at=session.created_at,
841     )
842
843
844 def _row_to_source(row: aiosqlite.Row) -> Source:
845     return Source(

```



```

846         id=row["id"],
847         account_id=row["account_id"],
848         provider=row["provider"],
849         provider_source_id=row["provider_source_id"],
850         title=row["title"],
851         kind=row["kind"],
852         watched=bool(row["watched"]),
853         seen_first_at=row["seen_first_at"],
854         last_seen_at=row["last_seen_at"],
855     )
856
857
858 def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
859     return ContentItem(
860         id=row["id"],
861         account_id=row["account_id"],
862         source_id=row["source_id"],
863         provider=row["provider"],
864         provider_message_id=row["provider_message_id"],
865         content_type=row["content_type"],
866         text=row["text"],
867         link_url=row["link_url"],
868         media_ref=row["media_ref"],
869         document_id=row["document_id"],
870         duration_s=row["duration_s"],
871         width=row["width"],
872         height=row["height"],
873         size_bytes=row["size_bytes"],
874         mime_type=row["mime_type"],
875         dedup_key=row["dedup_key"],
876         matched=bool(row["matched"]),
877         seen_at=row["seen_at"],
878     )
879
880
881 def _row_to_policy(row: aiosqlite.Row) -> Policy:
882     return Policy(
883         id=row["id"],
884         account_id=row["account_id"],
885         name=row["name"],
886         config_json=row["config_json"],
887         is_default=bool(row["is_default"]),
888         created_at=row["created_at"],
889         updated_at=row["updated_at"],
890     )
891
892
893 def _row_to_destination(row: aiosqlite.Row) -> Destination:
894     return Destination(
895         id=row["id"],
896         account_id=row["account_id"],
897         provider=row["provider"],
898         target=row["target"],
899         kind=row["kind"],
900         title=row["title"],
901         is_output=bool(row["is_output"]),
902         created_at=row["created_at"],
903     )
904
905
906 def _row_to_routing_rule(row: aiosqlite.Row) -> RoutingRule:
907     return RoutingRule(
908         id=row["id"],
909         account_id=row["account_id"],
910         source_id=row["source_id"],

```

```

911         policy_id=row["policy_id"],
912         destination_id=row["destination_id"],
913         enabled=bool(row["enabled"]),
914         priority=row["priority"],
915         created_at=row["created_at"],
916     )
917
918
919 def _row_to_delivery_attempt(row: aiosqlite.Row) -> DeliveryAttempt:
920     return DeliveryAttempt(
921         id=row["id"],
922         content_item_id=row["content_item_id"],
923         destination_id=row["destination_id"],
924         route_id=row["route_id"],
925         status=DeliveryStatus(row["status"]),
926         attempt_count=row["attempt_count"],
927         last_error=row["last_error"],
928         created_at=row["created_at"],
929         delivered_at=row["delivered_at"],
930         next_retry_at=row["next_retry_at"],
931     )
932
933
934 # --- account + session repos (ADR 0010) -----
935
936
937 async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
938     """Insert or update an account row; returns the persisted Account."""
939     await conn.execute(
940         """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
941         VALUES (?, ?, ?, ?, ?)
942         ON CONFLICT(id) DO UPDATE SET
943             telegram_id=excluded.telegram_id,
944             provider=excluded.provider,
945             display_name=excluded.display_name""",
946         (
947             account.id,
948             account.telegram_id,
949             account.provider,
950             account.display_name,
951             account.created_at.isoformat(),
952         ),
953     )
954     await conn.commit()
955     return await get_account(conn, account.id) # type: ignore[return-value]
956
957
958 async def get_account(conn: aiosqlite.Connection, account_id: int) -> Account | None:
959     conn.row_factory = aiosqlite.Row
960     async with conn.execute("SELECT * FROM accounts WHERE id = ?", (account_id,)) as cur:
961         row = await cur.fetchone()
962         return _row_to_account(row) if row else None
963
964
965 async def get_account_by_telegram_id(
966     conn: aiosqlite.Connection, telegram_id: int
967 ) -> Account | None:
968     """Resolve a Bot API ``from.id`` (or userbot account id) to an Account."""
969     conn.row_factory = aiosqlite.Row
970     async with conn.execute("SELECT * FROM accounts WHERE telegram_id = ?",
971                             (telegram_id,)) as cur:
972         row = await cur.fetchone()
973         return _row_to_account(row) if row else None

```

```

974
975 async def upsert_telegram_session(
976     conn: aiosqlite.Connection, session: TelegramSession
977 ) -> TelegramSession:
978     """Insert or update a telegram_sessions row; returns the persisted session."""
979     await conn.execute(
980         """INSERT INTO telegram_sessions
981             (account_id, phone, session_path, enabled, created_at, last_login_at)
982             VALUES (?, ?, ?, ?, ?, ?)
983             ON CONFLICT(account_id) DO UPDATE SET
984                 phone=excluded.phone,
985                 session_path=excluded.session_path,
986                 enabled=excluded.enabled,
987                 last_login_at=excluded.last_login_at""",
988         (
989             session.account_id,
990             session.phone,
991             session.session_path,
992             int(session.enabled),
993             session.created_at.isoformat(),
994             session.last_login_at.isoformat() if session.last_login_at else None,
995         ),
996     )
997     await conn.commit()
998     conn.row_factory = aiosqlite.Row
999     async with conn.execute(
1000         "SELECT * FROM telegram_sessions WHERE account_id = ?", (session.account_id,)
1001     ) as cur:
1002         row = await cur.fetchone()
1003         # The row must exist ? we just upserted it by PK (account_id). Assert the
1004         # invariant explicitly so mypy can narrow Row | None ? Row without a
1005         # type-ignore comment masking a real future regression.
1006         assert row is not None
1007         return _row_to_telegram_session(row)
1008
1009
1010 async def get_telegram_session(
1011     conn: aiosqlite.Connection, account_id: int
1012 ) -> TelegramSession | None:
1013     conn.row_factory = aiosqlite.Row
1014     async with conn.execute(
1015         "SELECT * FROM telegram_sessions WHERE account_id = ?", (account_id,)
1016     ) as cur:
1017         row = await cur.fetchone()
1018         return _row_to_telegram_session(row) if row else None
1019
1020
1021 async def list_runnable_sessions(
1022     conn: aiosqlite.Connection,
1023 ) -> list[tuple[Account, TelegramSession]]:
1024     """Return (account, session) pairs for sessions the manager should start.
1025
1026     This is the ADR 0010 structural rule: ``manager.run`` iterates this list,
1027     not ``accounts``. A bot-only user has no ``telegram_sessions`` row and is
1028     therefore inherently invisible to the Telethon supervisor ? the "don't
1029     start a client for bot-only users" invariant is enforced by query shape.
1030     """
1031     conn.row_factory = aiosqlite.Row
1032     async with conn.execute(
1033         """SELECT a.*, s.phone AS s_phone, s.session_path, s.enabled AS s_enabled,
1034             s.created_at AS s_created_at, s.last_login_at
1035             FROM accounts a
1036             JOIN telegram_sessions s ON s.account_id = a.id
1037             WHERE s.enabled = 1"""
1038     ) as cur:

```

```

1039         rows = await cur.fetchall()
1040     return [
1041         (
1042             _row_to_account(r),
1043             TelegramSession(
1044                 account_id=r["id"],
1045                 phone=r["s_phone"],
1046                 session_path=r["session_path"],
1047                 enabled=bool(r["s_enabled"]),
1048                 created_at=r["s_created_at"],
1049                 last_login_at=r["last_login_at"],
1050             ),
1051         )
1052     for r in rows
1053 ]
1054
1055
1056 # --- legacy User shims (backwards-compatible view over accounts + sessions) --
1057
1058
1059 async def upsert_user(conn: aiomysqlite.Connection, user: User) -> User:
1060     """Deprecated shim: upsert the (account, session) pair implied by a ``User``.
1061
1062     Maps the legacy v1 ``User`` onto the ADR 0010 schema: ``User.id`` becomes
1063     both ``accounts.id`` and ``accounts.telegram_id`` (legacy users are
1064     Telegram userbot accounts), and a ``telegram_sessions`` row is created so
1065     the manager can run them. ``User.enabled`` maps to ``session.enabled``.
1066     """
1067     account = Account(id=user.id, telegram_id=user.id, created_at=user.created_at)
1068     await upsert_account(conn, account)
1069     await upsert_telegram_session(
1070         conn,
1071         TelegramSession(
1072             account_id=user.id,
1073             phone=user.phone,
1074             session_path=f"user_{user.id}.session",
1075             enabled=user.enabled,
1076             created_at=user.created_at,
1077         ),
1078     )
1079     return await get_user(conn, user.id) # type: ignore[return-value]
1080
1081
1082 async def get_user(conn: aiomysqlite.Connection, user_id: int) -> User | None:
1083     """Deprecated shim: project the (account, session) pair back to a ``User``."""
1084     account = await get_account(conn, user_id)
1085     if account is None:
1086         return None
1087     session = await get_telegram_session(conn, user_id)
1088     return _account_session_to_user(account, session)
1089
1090
1091 async def list_enabled_users(conn: aiomysqlite.Connection) -> list[User]:
1092     """Deprecated shim: list users that have an enabled MTPROTO session.
1093
1094     Equivalent to projecting :func:`list_runnable_sessions` onto the legacy
1095     ``User`` shape. Bot-only accounts (no session) are excluded ? matching the
1096     pre-ADR-0010 "list enabled users" semantics where every row was runnable.
1097     """
1098     pairs = await list_runnable_sessions(conn)
1099     return [_account_session_to_user(a, s) for a, s in pairs]
1100
1101
1102 # --- Source repos (ADR 0011) -----
1103

```

```

1104
1105 async def upsert_source(conn: aiomysqlite.Connection, source: Source) -> Source:
1106     """Insert a source row, or refresh title/last_seen_at on conflict.
1107
1108     The conflict key is `(account_id, provider, provider_source_id)` per the
1109     ADR 0011 schema. Returns the persisted row.
1110
1111     ``watched`` is **user-controlled state** (toggled only via
1112     :func:`set_source_watched` / the ``/watch`` and ``/unwatch`` commands) and is
1113     therefore **never overwritten on conflict** ? the ingestion path
1114     (``upsert_channel`` / ``insert_video``) re-touches a source on every message,
1115     so clobbering ``watched`` here would silently un-watch a channel the user
1116     opted into after its first ingested video (ADR 0009). ``watched`` is only set
1117     on the initial INSERT of a new row. ``title`` is refreshed but never nulled:
1118     the ingestion shims pass ``title=None``, so ``COALESCE`` keeps the existing
1119     title rather than erasing the channel name.
1120     """
1121     await conn.execute(
1122         """INSERT INTO sources
1123             (account_id, provider, provider_source_id, title, kind, watched,
1124              seen_first_at, last_seen_at)
1125             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1126             ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
1127                 title=COALESCE(excluded.title, sources.title),
1128                 last_seen_at=excluded.last_seen_at""",
1129         (
1130             source.account_id,
1131             source.provider,
1132             source.provider_source_id,
1133             source.title,
1134             source.kind,
1135             int(source.watched),
1136             source.seen_first_at.isoformat(),
1137             (source.last_seen_at.isoformat() if source.last_seen_at else None),
1138         ),
1139     )
1140     await conn.commit()
1141     conn.row_factory = aiomysqlite.Row
1142     async with conn.execute(
1143         """SELECT * FROM sources
1144             WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1145         (source.account_id, source.provider, source.provider_source_id),
1146     ) as cur:
1147         row = await cur.fetchone()
1148         assert row is not None # just upserted by the conflict key
1149         return _row_to_source(row)
1150
1151
1152 async def get_source_by_provider_id(
1153     conn: aiomysqlite.Connection, account_id: int, provider: str, provider_source_id: str
1154 ) -> Source | None:
1155     """Resolve a source by its provider-native id."""
1156     conn.row_factory = aiomysqlite.Row
1157     async with conn.execute(
1158         """SELECT * FROM sources
1159             WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1160         (account_id, provider, provider_source_id),
1161     ) as cur:
1162         row = await cur.fetchone()
1163         return _row_to_source(row) if row else None
1164
1165
1166 # --- ContentItem repos (ADR 0011) -----
1167
1168

```

```

1169     async def insert_content_item(conn: aiomysqlite.Connection, item: ContentItem) ->
ContentItem:
1170         """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
1171
1172         The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
1173         unique constraints guard against duplicates:
1174         - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
1175         - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1176         """
1177         cur = await conn.execute(
1178             """INSERT INTO content_items
1179                 (account_id, source_id, provider, provider_message_id, content_type,
1180                 text, link_url, media_ref, document_id, duration_s, width, height,
1181                 size_bytes, mime_type, dedup_key, matched, seen_at)
1182             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
1183             (
1184                 item.account_id,
1185                 item.source_id,
1186                 item.provider,
1187                 item.provider_message_id,
1188                 item.content_type,
1189                 item.text,
1190                 item.link_url,
1191                 item.media_ref,
1192                 item.document_id,
1193                 item.duration_s,
1194                 item.width,
1195                 item.height,
1196                 item.size_bytes,
1197                 item.mime_type,
1198                 item.dedup_key,
1199                 int(item.matched),
1200                 item.seen_at.isoformat(),
1201             ),
1202         )
1203         await conn.commit()
1204         return item.model_copy(update={"id": cur.lastrowid})
1205
1206
1207     async def get_content_item_by_message(
1208         conn: aiomysqlite.Connection,
1209         account_id: int,
1210         source_id: int,
1211         provider_message_id: str,
1212     ) -> ContentItem | None:
1213         """Tier-0 lookup by the (account, source, provider-message-id) natural key."""
1214         conn.row_factory = aiomysqlite.Row
1215         async with conn.execute(
1216             """SELECT * FROM content_items
1217                 WHERE account_id = ? AND source_id = ? AND provider_message_id = ?""",
1218             (account_id, source_id, provider_message_id),
1219         ) as cur:
1220             row = await cur.fetchone()
1221             return _row_to_content_item(row) if row else None
1222
1223
1224     async def get_content_item_by_dedup_key(
1225         conn: aiomysqlite.Connection, account_id: int, dedup_key: str
1226     ) -> ContentItem | None:
1227         """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).
1228
1229         Replaces the Tier-1 (``document_id``) and Tier-2 (fingerprint) lookups with
1230         a single query: the ``dedup_key`` already encodes whichever tier was
1231         applicable at ingestion time.
1232         """

```

```

1233     conn.row_factory = aiosqlite.Row
1234     async with conn.execute(
1235         "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
1236         (account_id, dedup_key),
1237     ) as cur:
1238         row = await cur.fetchone()
1239     return _row_to_content_item(row) if row else None
1240
1241
1242 # --- Policy repos (ADR 0011, Option 2A) -----
1243
1244
1245 async def upsert_policy(conn: aiosqlite.Connection, policy: Policy) -> Policy:
1246     """Insert or update a policy row by ``(account_id, name)``.
1247
1248     Exactly one policy per account should carry ``is_default=True``; callers
1249     are responsible for flipping the previous default before flipping a new
1250     one (enforced by a partial unique index landed by future work, not in the
1251     schema ? SQLite ``CHECK`` can't express "at most one per group").
1252     """
1253     await conn.execute(
1254         """INSERT INTO policies (account_id, name, config_json, is_default,
1255                                created_at, updated_at)
1256                                VALUES (?, ?, ?, ?, ?, ?)
1257                                ON CONFLICT(account_id, name) DO UPDATE SET
1258                                config_json=excluded.config_json,
1259                                is_default=excluded.is_default,
1260                                updated_at=excluded.updated_at""",
1261         (
1262             policy.account_id,
1263             policy.name,
1264             policy.config_json,
1265             int(policy.is_default),
1266             policy.created_at.isoformat(),
1267             policy.updated_at.isoformat(),
1268         ),
1269     )
1270     await conn.commit()
1271     conn.row_factory = aiosqlite.Row
1272     async with conn.execute(
1273         "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1274         (policy.account_id, policy.name),
1275     ) as cur:
1276         row = await cur.fetchone()
1277     assert row is not None
1278     return _row_to_policy(row)
1279
1280
1281 async def get_default_policy(conn: aiosqlite.Connection, account_id: int) -> Policy |
None:
1282     """Return the account's default policy (``is_default=1``)."""
1283     conn.row_factory = aiosqlite.Row
1284     async with conn.execute(
1285         "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1286         (account_id,),
1287     ) as cur:
1288         row = await cur.fetchone()
1289     return _row_to_policy(row) if row else None
1290
1291
1292 def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1293     """Decode a policy config JSON object.
1294
1295     ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
1296     expected to be a JSON object so legacy flat ``FilterConfig`` keys and future

```

```

1297     nested stage config can coexist.
1298     """
1299     decoded = json.loads(raw)
1300     if not isinstance(decoded, dict):
1301         raise ValueError("policy config_json must decode to a JSON object")
1302     return decoded
1303
1304
1305 def _policy_config_dict_to_json(config: dict[str, object]) -> str:
1306     """Encode policy config deterministically and compactly."""
1307     return json.dumps(config, separators=(",", ":"), sort_keys=True)
1308
1309
1310 async def load_policy_config(conn: aiomysqlite.Connection, account_id: int) -> dict[str,
object]:
1311     """Load the raw default policy config as a JSON object.
1312
1313     Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
1314     as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
1315     through the flat ``FilterConfig`` model.
1316     """
1317     policy = await get_default_policy(conn, account_id)
1318     if policy is None:
1319         return FilterConfig().model_dump(mode="json")
1320     return _policy_config_json_to_dict(policy.config_json)
1321
1322
1323 async def save_policy_config(
1324     conn: aiomysqlite.Connection, account_id: int, config: dict[str, object]
1325 ) -> None:
1326     """Persist the raw default policy config object."""
1327     existing = await get_default_policy(conn, account_id)
1328     await upsert_policy(
1329         conn,
1330         Policy(
1331             id=existing.id if existing is not None else None,
1332             account_id=account_id,
1333             name="default",
1334             config_json=_policy_config_dict_to_json(config),
1335             is_default=True,
1336             created_at=(existing.created_at if existing is not None else _now()),
1337             updated_at=_now(),
1338         ),
1339     )
1340
1341
1342 # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
1343
1344
1345 async def upsert_destination(conn: aiomysqlite.Connection, dest: Destination) ->
Destination:
1346     """Insert or update a destination row by (account, provider, target)."""
1347     await conn.execute(
1348         """INSERT INTO destinations
1349             (account_id, provider, target, kind, title, is_output, created_at)
1350             VALUES (?, ?, ?, ?, ?, ?, ?)
1351             ON CONFLICT(account_id, provider, target) DO UPDATE SET
1352                 kind=excluded.kind,
1353                 title=excluded.title,
1354                 is_output=excluded.is_output""",
1355         (
1356             dest.account_id,
1357             dest.provider,
1358             dest.target,
1359             dest.kind,

```



```

1360         dest.title,
1361         int(dest.is_output),
1362         dest.created_at.isoformat(),
1363     ),
1364 )
1365 await conn.commit()
1366 conn.row_factory = aiosqlite.Row
1367 async with conn.execute(
1368     """SELECT * FROM destinations
1369         WHERE account_id = ? AND provider = ? AND target = ?""",
1370     (dest.account_id, dest.provider, dest.target),
1371 ) as cur:
1372     row = await cur.fetchone()
1373     assert row is not None
1374     return _row_to_destination(row)
1375
1376
1377 async def get_destination(
1378     conn: aiosqlite.Connection, account_id: int, provider: str, target: str
1379 ) -> Destination | None:
1380     """Resolve a destination by its (account, provider, target) natural key."""
1381     conn.row_factory = aiosqlite.Row
1382     async with conn.execute(
1383         "SELECT * FROM destinations WHERE account_id = ? AND provider = ? AND target = ?"
1384         ,
1385         (account_id, provider, target),
1386     ) as cur:
1387         row = await cur.fetchone()
1388         return _row_to_destination(row) if row else None
1389

```

5. assistant (2026-07-06T20:52:45.283Z)

Now let me read the tests and check the callers of these functions to identify behavior changes.

6. user (2026-07-06T20:52:46.568Z)

```

1     """Tests for the SQLite data layer (db.py) using an in-memory database."""
2
3     from __future__ import annotations
4
5     import sqlite3
6
7     import aiosqlite
8     import pytest
9
10    from tg_video_filter import db
11    from tg_video_filter.models import (
12        DEFAULT_FILTER_CONFIG,
13        Account,
14        Channel,
15        ContentItem,
16        DeliveryStatus,
17        Destination,
18        FilterConfig,
19        Policy,
20        Source,
21        TelegramSession,
22        User,
23        VideoRecord,
24    )
25
26    USER_ID = 111
27    CHANNEL_ID = -1001234567890

```

```

28
29
30 @pytest.fixture
31 async def conn() -> aisqlite.Connection:
32     """An in-memory SQLite connection with the schema applied."""
33     c = await aisqlite.connect(":memory:")
34     await db._apply_schema(c)
35     yield c
36     await c.close()
37
38
39 async def _seed_user(conn: aisqlite.Connection, uid: int = USER_ID) -> User:
40     return await db.upsert_user(conn, User(id=uid, phone="+15555550100"))
41
42
43 # --- schema / init -----
44
45
46 async def test_init_db_creates_tables(tmp_path: object) -> None:
47     db_path = str(tmp_path) + "/test.db" # type: ignore[operator]
48     await db.init_db(db_path)
49     # Re-open and confirm all expected tables exist. Post-ADR-0011 the
50     # provider-neutral tables are sources/content_items/policies (plus the
51     # empty Track-C tables destinations/routing_rules/delivery_attempts);
52     # the legacy channels/videos/filter_config are absent on fresh DBs
53     # (they only exist as *_deprecated on migrated legacy DBs).
54     async with (
55         aisqlite.connect(db_path) as c,
56         c.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name") as
cur,
57     ):
58         names = [r[0] for r in await cur.fetchall()]
59         assert set(names) >= {
60             "accounts",

```

7. user (2026-07-06T20:52:47.822Z)

```

C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\manager.py:140:         cfg = await
db.load_filter_config(conn, session.account_id)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\models.py:193:     ``load_filter_config`` /
``save_filter_config`` shims read/write that row.
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:619:     #     (Option 2A).
is_default=1 so the legacy load_filter_config shim reads
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1310:async def
load_policy_config(conn: aisqlite.Connection, account_id: int) -> dict[str, object]:
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1313:     Unlike
:func:`load_filter_config`, this preserves future ADR 0012 keys such
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1323:async def save_policy_config(
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1749:async def
load_filter_config(conn: aisqlite.Connection, user_id: int) -> FilterConfig:
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1762:async def
save_filter_config(conn: aisqlite.Connection, user_id: int, cfg: FilterConfig) -> None:
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1778:     await
save_policy_config(conn, user_id, raw_config)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1845:     "load_policy_config",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1846:     "save_policy_config",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1864:     "load_filter_config",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1865:     "save_filter_config",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\command_service.py:465:         await
db.save_filter_config(conn, user_id, cfg)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\commands.py:78:         current_cfg = await
db.load_filter_config(conn, user.id)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\commands.py:82:         new_cfg = await
db.load_filter_config(conn, user.id)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\commands.py:112:     cfg = await

```



```

1807         else None
1808     ),
1809     duration_s=r["duration_s"],
1810     width=r["width"],
1811     height=r["height"],
1812     size_bytes=r["size_bytes"],
1813     mime_type=r["mime_type"],
1814     matched=bool(r["matched"]),
1815     forwarded=False,
1816     seen_at=r["seen_at"],
1817 )
1818 for r in rows
1819 ]
1820
1821
1822 __all__ = [
1823     "init_db",
1824     "compute_dedup_key",
1825     "_safe_int_cast",
1826     # Account + session repos (ADR 0010)
1827     "upsert_account",
1828     "get_account",
1829     "get_account_by_telegram_id",
1830     "upsert_telegram_session",
1831     "get_telegram_session",
1832     "list_runnable_sessions",
1833     # Provider-neutral domain repos (ADR 0011)
1834     "upsert_source",
1835     "get_source_by_provider_id",
1836     "get_source_by_id",
1837     "set_source_watched",
1838     "list_account_sources",
1839     "should_ingest_source",
1840     "insert_content_item",
1841     "get_content_item_by_message",
1842     "get_content_item_by_dedup_key",
1843     "upsert_policy",
1844     "get_default_policy",
1845     "load_policy_config",
1846     "save_policy_config",
1847     # Destination / Routing / Delivery repos (ADR 0011, Track C)
1848     "upsert_destination",

```

9. assistant (2026-07-06T20:52:54.891Z)

Now let me look at the callers, especially `commands.py` which catches exceptions around `load_filter_config`, and the `FilterConfig` model to understand `model_dump` shape.

10. user (2026-07-06T20:52:56.904Z)

```

60     async def on_command(event: object) -> None:
61         message = getattr(event, "message", None)
62         if message is None:
63             return
64         if not getattr(message, "out", False):
65             return
66         peer = getattr(message, "peer_id", None)
67         peer_user_id = getattr(peer, "user_id", None) if peer else None
68         if peer_user_id != user.id:
69             return
70
71         text: str = getattr(message, "message", "") or ""
72         client: TelegramClient = getattr(event, "client", None) # type:
ignore[assignment]
73

```

```

74         # Defer DB load ? skip non-command self-messages (review finding #2).
75         if not text.startswith("/"):
76             return
77
78         current_cfg = await db.load_filter_config(conn, user.id)
79         result: CommandResult = await svc.handle(user.id, text, conn, current_cfg)
80
81         if result.config_changed:
82             new_cfg = await db.load_filter_config(conn, user.id)
83             engine_holder[ENGINE_KEY] = FilterEngine(new_cfg)
84
85         if result.reply_text:
86             await _reply(client, message, result.reply_text)
87
88     return on_command
89
90
91 # --- backwards-compat helpers (test_commands.py references) -----
92
93
94 async def _update_config(
95     conn: aiosqlite.Connection,
96     user_id: int,
97     max_size_mb: int | None = None,
98     max_duration_s: int | None = None,
99     min_duration_s: int | None = None,
100     target_height: int | None = None,
101     exclude_gif: bool | None = None,
102     exclude_round: bool | None = None,
103     exclude_nosound: bool | None = None,
104 ) -> FilterConfig:
105     """Deprecated wrapper ? delegates to ``CommandService._update``.
106
107     Kept for backwards-compatibility with ``tests/test_commands.py`` which
108     imports ``_update_config`` from here. New code should call
109     :meth:`CommandService.handle` which returns a :class:`CommandResult`.
110     """
111     svc = CommandService()
112     cfg = await db.load_filter_config(conn, user_id)
113     return await svc._update(
114         conn,
115         user_id,
116         cfg,
117         max_size_mb=max_size_mb,
118         max_duration_s=max_duration_s,
119         min_duration_s=min_duration_s,
120         target_height=target_height,
121         exclude_gif=exclude_gif,
122         exclude_round=exclude_round,
123         exclude_nosound=exclude_nosound,
124     )
125
126
127 async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
128     try:
129         await client.send_message("me", text)

```

11. user (2026-07-06T20:52:57.467Z)

```

150     """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152     Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153     (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154     NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155     ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup

```

```

156 wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157 """"
158
159 model_config = ConfigDict(frozen=True)
160
161 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162 account_id: int = Field(description="FK ? accounts.id")
163 source_id: int = Field(description="FK ? sources.id")
164 provider: str = Field(default="telegram", description="Provider namespace")
165 provider_message_id: str = Field(
166     description="Provider-native message id (Telegram message_id, ?)"
167 )
168 content_type: str = Field(default="video", description="'video' | 'link' |
'article'")
169 text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170 link_url: str | None = Field(default=None, description="For link/article posts
(Phase 1: NULL)")
171 media_ref: str | None = Field(default=None, description="Provider media reference")
172 document_id: str | None = Field(
173     default=None,
174     description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175 )
176 duration_s: int | None = None
177 width: int | None = None
178 height: int | None = None
179 size_bytes: int | None = None
180 mime_type: str | None = None
181 dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182 matched: bool = False
183 seen_at: datetime = Field(default_factory=_utcnow)
184
185
186 class Policy(BaseModel):
187     """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189     Generalizes the legacy per-account ``filter_config`` row so one account can
190     own multiple policies (enables ADR 0009's per-route filter overrides) and so
191     future enrichment/security stages grow into the same ``config_json`` column.
192     Exactly one policy per account carries ``is_default=True``; the legacy
193     ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194     """
195
196     model_config = ConfigDict(frozen=True)
197
198     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199     account_id: int = Field(description="FK ? accounts.id")
200     name: str = Field(description="Policy name; 'default' for the account default
policy")
201     config_json: str = Field(
202         description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203     )
204     is_default: bool = Field(
205         default=False, description="Exactly one per account (UNIQUE partial index)"
206     )
207     created_at: datetime = Field(default_factory=_utcnow)
208     updated_at: datetime = Field(default_factory=_utcnow)
209
210
211 # --- Track C: destinations, routing, delivery state (ADR 0011) -----
212 #
213 # These tables were created empty by Track B (_SCHEMA); Track C populates
214 # destinations (backfilled from filter_config_deprecated.config_json), writes
215 # delivery_attempts (replacing the legacy videos.forwarded boolean), and

```

```

216 # defines the models for per-destination delivery state. The source/destination
217 # invariant (a channel is either source or destination, never both) is enforced
218 # at the app level by future selective-scanning logic (Track E, #30).
219
220
221 class Destination(BaseModel):
222     """A delivery target ? where matched content is sent (ADR 0011, Track C).
223
224     Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
225     can have multiple named output channels. Backfilled from the existing
226     per-account ``delivery_target``; the default destination is the target used
227     when no explicit routing rule matches.
228     """
229

```

12. assistant (2026-07-06T20:53:00.019Z)

Let me look at the FilterConfig model definition and command_service save path, plus test_db regression test.

13. user (2026-07-06T20:53:02.137Z)

```

224: Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
225- can have multiple named output channels. Backfilled from the existing
226: per-account ``delivery_target``; the default destination is the target used
227- when no explicit routing rule matches.
228- """
229-
230- model_config = ConfigDict(frozen=True)
231-
232- id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
233- account_id: int = Field(description="FK ? accounts.id")
234- provider: str = Field(default="telegram", description="Provider namespace")
235- target: str = Field(description="Telethon entity: 'me', '@username', or chat id")
236- kind: str = Field(default="saved", description="'saved' | 'channel' | 'chat'")
237- title: str | None = None
238- is_output: bool = Field(
239-     default=False,
240-     description="ADR 0009: distinguishes output channels from ingest sources",
241- )
242- created_at: datetime = Field(default_factory=_utcnow)
243-
244-
245- class RoutingRule(BaseModel):
246-     """A rule that maps a source through a policy to a destination (ADR 0009).
247-
248-     Canonical name: ``routing_rules`` (locked in umbrella issue #25). When
249-     ``policy_id`` is NULL, the account's default policy is used. When multiple
250-     rules match, the highest ``priority`` wins.
251-     """
252-
253-     model_config = ConfigDict(frozen=True)
254-
255-     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
256-     account_id: int = Field(description="FK ? accounts.id")
257-     source_id: int = Field(description="FK ? sources.id")
258-     policy_id: int | None = Field(
259-         default=None,
260-         description="FK ? policies.id; NULL means use account default policy",
261-     )
262-     destination_id: int = Field(description="FK ? destinations.id")
263-     enabled: bool = Field(default=True)
264-     priority: int = Field(default=0, description="Higher wins when multiple rules match")
265-     created_at: datetime = Field(default_factory=_utcnow)
266-

```

```
--
299: class FilterConfig(BaseModel):
300-     """Per-user filter configuration.
301-
302-     v1 rules: duration bounds, minimum height (resolution), and max file size.
303-     Stored in the DB as a JSON blob (`filter_config.config_json`) so adding a
304-     new field later is backwards-compatible.
305-     """
306-
307-     # Duration bounds in seconds. None means unbounded on that side.
308-     min_duration_s: int | None = Field(default=5, ge=0)
309-     max_duration_s: int | None = Field(default=7200, ge=0)
310-
311-     # Resolution: a video passes if its height >= target_height.
312-     # None disables the resolution rule.
313-     target_height: int | None = Field(default=1080, ge=1)
314-
```

[Showing results with pagination = limit: 60]

14. user (2026-07-06T20:53:05.299Z)

```
299     class FilterConfig(BaseModel):
300         """Per-user filter configuration.
301
302         v1 rules: duration bounds, minimum height (resolution), and max file size.
303         Stored in the DB as a JSON blob (`filter_config.config_json`) so adding a
304         new field later is backwards-compatible.
305         """
306
307         # Duration bounds in seconds. None means unbounded on that side.
308         min_duration_s: int | None = Field(default=5, ge=0)
309         max_duration_s: int | None = Field(default=7200, ge=0)
310
311         # Resolution: a video passes if its height >= target_height.
312         # None disables the resolution rule.
313         target_height: int | None = Field(default=1080, ge=1)
314
315         # Max file size in mebibytes. None disables the size rule.
316         max_size_mb: int | None = Field(default=2048, ge=1)
317
318         # Content-type exclusions. GIFs and round videos are excluded by default
319         # (their duration/dimensions are usually incidental to real video filters);
320         # silent videos are off by default since some legit clips have no audio.
321         # Existing users' stored configs are unaffected ? the DB JSON blob overrides
322         # these defaults, so this only changes the behaviour for fresh installs.
323         exclude_gif: bool = Field(
324             default=True,
325             description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
326         )
327         exclude_round: bool = Field(
328             default=True,
329             description="If True, exclude circular video messages (round_message=True)",
330         )
331         exclude_nosound: bool = Field(
332             default=False,
333             description="If True, exclude videos without an audio track (nosound=True)",
334         )
335
336         # How matched videos are delivered: forward the video, send a link, or both.
337         # Stored in the same JSON blob, so adding this field is backwards-compatible
338         # (existing rows without it default to FORWARD).
339         delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
340
341         # Where matched videos / link messages are sent. Accepts:
```



```

342 # - "me" (default): Saved Messages ? exact v1.1 behaviour.
343 # - "@username": a public channel the account is a member of.
344 # - "-100..." or a bare int-as-string: a numeric Telegram chat id
345 # (private channels without a username).
346 # The string is passed verbatim to Telethon's forward_messages / send_message.
347 # See ADR 0005 for rationale and access requirements.
348 delivery_target: str = Field(
349     default="me",
350     min_length=1,
351     description="Telethon entity to deliver to: 'me', '@username', or a chat id",
352 )
353
354 @field_validator("delivery_target", mode="before")
355 @classmethod
356 def _coerce_delivery_target(cls, v: object) -> str:
357     """Accept int chat ids (e.g. -1001234567890) and coerce to str.
358

```

15. assistant (2026-07-06T20:53:07.801Z)

Now I have a good understanding. Let me check for a `DeliveryMode` enum serialization concern and look at the regression test in test_db.py, then run repros.

16. user (2026-07-06T20:53:09.356Z)

```

460: await db.save_filter_config(conn, USER_ID, custom)
465:async def test_save_filter_config_overwrites(conn: aiosqlite.Connection) -> None:
467:     await db.save_filter_config(conn, USER_ID, FilterConfig(max_duration_s=100))
468:     await db.save_filter_config(conn, USER_ID, FilterConfig(max_duration_s=200))
473:async def test_save_filter_config_preserves_unknown_policy_keys(
478:     await db.save_policy_config(
484:         "topics": {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"},
485:         "security_checks": {"blocked_domains": ["sketchy.example"]},
486:         "deep_inspect": True,
490:     await db.save_filter_config(conn, USER_ID, FilterConfig(max_size_mb=512))
492:     raw = await db.load_policy_config(conn, USER_ID)
494:     assert raw["topics"] == {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"}
495:     assert raw["security_checks"] == {"blocked_domains": ["sketchy.example"]}
496:     assert raw["deep_inspect"] is True
507:     await db.save_filter_config(conn, USER_ID, cfg)
521:         await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_mode=mode))
569:         await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_target=target))
762:         # Legacy data preserved in the deprecated tables.
1022:async def test_upsert_source_updates_title_but_preserves_watched_on_conflict(
1047:     assert updated.watched is True # watched preserved, not clobbered
1176:     """save_filter_config ? load_filter_config round-trips through policies."""
1179:     await db.save_filter_config(conn, USER_ID, custom)
1392:     - ALL rows are preserved (no silent drops ? fix #1 & #3).
1445:         # different message/channel. Fix #3 must preserve both.
1467:         assert orphan[0] is None # title unknown (no channels row)
1489:     ), "fix #3: collision loser not preserved at tg:msg:"

```

17. assistant (2026-07-06T20:53:12.177Z)

Now let me run repros. First, the key concern: `load\_policy\_config` default returns `FilterConfig().model\_dump(mode="json")` when no policy ? does that round-trip stable with what save writes (especially `delivery\_mode` enum)? And the exception-type change in `load\_filter\_config`. Let me write a repro script.

18. assistant (2026-07-06T20:53:27.316Z)

```

import asyncio, json
import aiosqlite
from datetime import UTC, datetime

```

```

from tg_video_filter import db
from tg_video_filter.models import Account, FilterConfig, Policy

async def mk():
    c = await aiosqlite.connect(":memory:")
    await db._apply_schema(c)
    await db.upsert_account(c, Account(id=1, telegram_id=1, created_at=datetime.now(UTC)))
    return c

async def main():
    # --- 1: load_policy_config default shape vs saved shape round-trip ---
    c = await mk()
    default_no_policy = await db.load_policy_config(c, 1)
    print("default (no policy):", json.dumps(default_no_policy, sort_keys=True))
    print(" delivery_mode type:", type(default_no_policy["delivery_mode"]).__name__,
repr(default_no_policy["delivery_mode"]))
    # Now save the SAME dict back and reload
    await db.save_policy_config(c, 1, default_no_policy)
    reloaded = await db.load_policy_config(c, 1)
    print("reloaded after save:", json.dumps(reloaded, sort_keys=True))
    print(" round-trip stable:", default_no_policy == reloaded)
    await c.close()

    # --- 2: load_filter_config exception type on corrupt config_json ---
    c = await mk()
    # Insert a default policy with non-object JSON (a bare string)
    await db.upsert_policy(c, Policy(id=None, account_id=1, name="default",
                                     config_json=json.dumps("not-an-object"),
                                     is_default=True,
                                     created_at=datetime.now(UTC),
updated_at=datetime.now(UTC)))
    print("\n--- corrupt (non-object JSON) ---")
    for fn_name, fn in (("load_filter_config", db.load_filter_config),
                        ("load_policy_config", db.load_policy_config)):
        try:
            await fn(c, 1)
            print(fn_name, "-> NO RAISE")
        except Exception as e:
            print(fn_name, "-> raises", type(e).__module__ + "." + type(e).__name__, ":", e)
    await c.close()

    # --- 3: malformed (invalid) JSON ---
    c = await mk()
    await db.upsert_policy(c, Policy(id=None, account_id=1, name="default",
                                     config_json="{not valid json",
                                     is_default=True,
                                     created_at=datetime.now(UTC),
updated_at=datetime.now(UTC)))
    print("\n--- malformed JSON ---")
    for fn_name, fn in (("load_filter_config", db.load_filter_config),
                        ("load_policy_config", db.load_policy_config)):
        try:
            await fn(c, 1)
            print(fn_name, "-> NO RAISE")
        except Exception as e:
            print(fn_name, "-> raises", type(e).__module__ + "." + type(e).__name__)
    # save_filter_config over a corrupt config -> does it drop unknown keys?
    await db.save_filter_config(c, 1, FilterConfig(max_size_mb=512))
    after = await db.load_policy_config(c, 1)
    print("save_filter_config over malformed -> config now:", json.dumps(after,
sort_keys=True)[:120])
    await c.close()

```

```

# --- 4: old behavior comparison: FilterConfig.model_validate_json on non-object ---
print("\n--- OLD path exception type (model_validate_json) ---")
try:
    FilterConfig.model_validate_json(json.dumps("not-an-object"))
except Exception as e:
    print("model_validate_json(non-object) ->", type(e).__module__ + "." + type(e).__name__)
try:
    FilterConfig.model_validate_json("{not valid json}")
except Exception as e:
    print("model_validate_json(malformed) ->", type(e).__module__ + "." + type(e).__name__)

asyncio.run(main())

```

19. user (2026-07-06T20:53:27.317Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

20. assistant (2026-07-06T20:53:29.806Z)

The file doesn't exist yet; the error is a false guard. Let me use a different filename.

21. user (2026-07-06T20:53:44.337Z)

```

default (no policy): {"delivery_mode": "forward", "delivery_target": "me", "exclude_gif": true,
"exclude_nosound": false, "exclude_round": true, "max_duration_s": 7200, "max_size_mb": 2048,
"min_duration_s": 5, "target_height": 1080}
  delivery_mode: str 'forward'
reloaded after save: {"delivery_mode": "forward", "delivery_target": "me", "exclude_gif": true,
"exclude_nosound": false, "exclude_round": true, "max_duration_s": 7200, "max_size_mb": 2048,
"min_duration_s": 5, "target_height": 1080}
  round-trip stable: True

--- corrupt (non-object JSON) ---
load_filter_config -> raises builtins.ValueError : policy config_json must decode to a JSON
object
load_policy_config -> raises builtins.ValueError : policy config_json must decode to a JSON
object

--- malformed JSON ---
load_filter_config -> raises json.decoder.JSONDecodeError
load_policy_config -> raises json.decoder.JSONDecodeError
save_filter_config over malformed -> config now: {"delivery_mode": "forward", "delivery_target":
"me", "exclude_gif": true, "exclude_nosound": false, "exclude_round": tr

--- OLD path exception type (model_validate_json) ---
model_validate_json(non-object) -> pydantic_core._pydantic_core.ValidationError
model_validate_json(malformed) -> pydantic_core._pydantic_core.ValidationError

```

22. assistant (2026-07-06T20:53:47.871Z)

Confirmed the exception-type change. Round-trip is stable. Now the critical question: does any caller catch the OLD `ValidationError`? Let me check `manager.py:140`, `commands.py`, and the command flow around load_filter_config for exception handling.

23. user (2026-07-06T20:53:48.896Z)

```

125         from tg_video_filter.telethon_client import build_client, make_new_message_handler
126
127         client = build_client(session, settings)
128         await client.connect()
129         if not await client.is_user_authorized():
130             # No valid session ? the onboarding step (Phase 5) must run first.
131             log.error(

```

```

132         "client_not_authorized account=%s; run `python -m tg_video_filter.bootstrap`
133     "
134     "to create the session",
135     session.account_id,
136 )
137 await client.disconnect()
138 return
139
140 async with aiosqlite.connect(settings.db_path) as conn:
141     cfg = await db.load_filter_config(conn, session.account_id)
142     # Mutable holder shared across all handlers; dict key swap is atomic
143     # under the GIL for single-threaded asyncio dispatch (see ADR 0007).
144     engine_holder: dict[str, FilterEngine] = {ENGINE_KEY: FilterEngine(cfg)}
145     # The legacy User view is reconstructed for handlers that still key on
146     # user.id (commands.py authz, backfill). These migrate to Account in
147     # Track D; for now the integer identity is identical.
148     user = User(id=session.account_id, phone=session.phone, enabled=session.enabled)
149     handler = make_new_message_handler(
150         user=user,
151         conn=conn,
152         engine_holder=engine_holder,
153         client=client,
154         delivery_mode=cfg.delivery_mode,
155         delivery_target=cfg.delivery_target,
156     )
157     client.add_event_handler(handler, events.NewMessage(incoming=True))
158
159     # Register the /backfill command handler on Saved Messages.
160     backfill_handler = make_backfill_command_handler(
161         client=client,
162         user=user,
163         conn=conn,
164         engine_holder=engine_holder,
165         delivery_mode=cfg.delivery_mode,

```

24. user (2026-07-06T20:53:50.300Z)

```

84:         try:
85:             # gather.return_exceptions=True so one session's failure doesn't tear
86:             results = await asyncio.gather(*tasks, return_exceptions=True)
87:         except asyncio.CancelledError:
88:             await asyncio.gather(*tasks, return_exceptions=True)
89:         try:

```

25. assistant (2026-07-06T20:53:55.740Z)

No caller catches `ValidationError` specifically. The exception-type change is low severity (both old and new raise, no caller narrows on the type). Now let me investigate the more impactful concern the prompt raised: the **asymmetry** between `save\_filter\_config` (catches corrupt ? `{}`, silently drops unknown keys) vs `load\_filter\_config`/`load\_policy\_config` (raise on corrupt).

The real question: on a corrupt config, `load\_filter\_config` raises at manager startup (line 140) ? the session start would crash. Let me verify the manager's per-session error handling and check whether a corrupt policy config can actually arise. Let me look at lines 84-92 and 178.

26. user (2026-07-06T20:53:57.013Z)

```

75         log.warning("manager_no_runnable_sessions; exiting")
76         return 0
77
78         tasks: list[asyncio.Task[None]] = [
79             asyncio.create_task(client_runner(session),
80             name=f"session-{session.account_id}")

```

```

80         for _account, session in pairs
81     ]
82     _install_signal_handlers(tasks)
83
84     try:
85         # gather.return_exceptions=True so one session's failure doesn't tear
86         # down the others; we log each outcome below.
87         results = await asyncio.gather(*tasks, return_exceptions=True)
88     except asyncio.CancelledError:
89         log.info("manager_cancelled; cancelling session tasks")
90         for t in tasks:
91             t.cancel()
92         await asyncio.gather(*tasks, return_exceptions=True)
93         raise
94
95     for (_account, session), result in zip(pairs, results, strict=True):
96         if isinstance(result, BaseException):
97             log.error(
98                 "client_crashed account=%s error=%r",
99                 session.account_id,
100                 result,
101                 exc_info=result,
102             )
103         else:
104             log.info("client_finished account=%s", session.account_id)

```

27. assistant (2026-07-06T20:54:02.979Z)

This is a pre-existing behavior (the old `load\_filter\_config` via `model\_validate\_json` also raised on corrupt/malformed JSON), so a corrupt config crashing the session is not a regression introduced by this PR. Both old and new raise. Not a new defect.

Now let me focus on the merge-order concern and the `command\_service.\_update` save path ? the most likely place for a real bug. The prompt asks: "What if the existing config_json has a key that is BOTH a FilterConfig field AND was customized (does /set correctly win)?" Let me examine `command\_service.py` `\_update` to see how `cfg` is built before save.

28. user (2026-07-06T20:54:04.112Z)

```

430         return s
431     return None
432
433     # -- internal -----
434
435     async def _update(
436         self,
437         conn: aiosqlite.Connection,
438         user_id: int,
439         current: FilterConfig,
440         max_size_mb: int | None = None,
441         max_duration_s: int | None = None,
442         min_duration_s: int | None = None,
443         target_height: int | None = None,
444         exclude_gif: bool | None = None,
445         exclude_round: bool | None = None,
446         exclude_nosound: bool | None = None,
447     ) -> FilterConfig:
448         """Apply partial updates to the account's config, persist, and return."""
449         updates: dict[str, object] = {}
450         if max_size_mb is not None:
451             updates["max_size_mb"] = max_size_mb
452         if max_duration_s is not None:
453             updates["max_duration_s"] = max_duration_s
454         if min_duration_s is not None:
455             updates["min_duration_s"] = min_duration_s

```

```

456         if target_height is not None:
457             updates["target_height"] = target_height
458         if exclude_gif is not None:
459             updates["exclude_gif"] = exclude_gif
460         if exclude_round is not None:
461             updates["exclude_round"] = exclude_round
462         if exclude_nosound is not None:
463             updates["exclude_nosound"] = exclude_nosound
464         cfg = current.model_copy(update=updates) if updates else current
465         await db.save_filter_config(conn, user_id, cfg)
466         log.info(
467             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
468             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
469             user_id,
470             cfg.max_size_mb,
471             cfg.min_duration_s,
472             cfg.max_duration_s,
473             cfg.target_height,
474             cfg.exclude_gif,

```

29. assistant (2026-07-06T20:54:13.676Z)

Now I see the full flow. `current` comes from `load\_filter\_config` which validates through `FilterConfig` ? meaning any *extra flat keys* stored in config_json that are NOT FilterConfig fields but that a user might have set are dropped from `current`. But nested pipeline keys (topics etc.) are preserved by the merge in save_filter_config because save reads the raw existing config (not `current`).

Let me test the key merge-precedence scenario and one subtle issue: what if a pipeline key has the **same name** as a FilterConfig field but different semantics. That's not realistic. Let me instead test the important asymmetry: **`load\_filter\_config` reads through FilterConfig, but the merge in `save\_filter\_config` overlays `cfg.model\_dump()` onto raw.** If a FilterConfig field was customized in raw AND the user's `/set` doesn't touch it, does the customized value survive? **Yes** because `current` is loaded from raw via FilterConfig. Let me verify the round-trip and also test a subtle bug: **extra flat unknown keys survive, but does `load\_filter\_config` silently drop them** (that's fine, they're not FilterConfig).

Let me now test the scenario the happy path may have missed: **does `save\_filter\_config` preserve unknown keys when starting from NO existing policy** (first-ever /set), and check the `DeliveryMode` enum serialization is a plain string (already confirmed "forward"). Let me run one more targeted repro on merge precedence.

30. user (2026-07-06T20:54:31.276Z)

```

loaded current: max_size_mb=512 max_duration_s=100
after /set target_height=720:
  max_size_mb: 512 (expect 512 preserved)
  max_duration_s: 100 (expect 100 preserved)
  target_height: 720 (expect 720)
  topics: {'include': ['AI']} (expect preserved)
  extra_flat_key: hello (UNKNOWN FLAT KEY - preserved?)

```

Scenario B first save: max_size_mb=256 (expect 256)

Scenario C sort_keys: before==after (data): False

31. assistant (2026-07-06T20:54:34.848Z)

Interesting ? Scenario C shows `before != after`. Let me dig into WHY. This could be a real data-loss finding, or just key reordering surfacing in the dict comparison. But dicts compare by content not order, so `before == after` being False means actual content changed. Let me investigate.

32. user (2026-07-06T20:54:43.918Z)

```
BEFORE: {"max_size_mb": 999, "min_duration_s": 5, "target_height": 1080, "topics": {"a": 2, "z": 1}}
AFTER : {"delivery_mode": "forward", "delivery_target": "me", "exclude_gif": true,
"exclude_nosound": false, "exclude_round": true, "max_duration_s": 7200, "max_size_mb": 999,
"min_duration_s": 5, "target_height": 1080, "topics": {"a": 2, "z": 1}}
```

```
Keys added by save: {'delivery_target', 'exclude_nosound', 'delivery_mode', 'max_duration_s',
'exclude_round', 'exclude_gif'}
```

```
Keys removed by save: set()
```

```
CHANGED delivery_mode : None -> forward
CHANGED delivery_target : None -> me
CHANGED exclude_gif : None -> True
CHANGED exclude_nosound : None -> False
CHANGED exclude_round : None -> True
CHANGED max_duration_s : None -> 7200
```

33. assistant (2026-07-06T20:54:53.293Z)

That difference is benign ? the save fills in default FilterConfig fields that were previously absent, and no topics/unknown keys were lost. This is expected and correct (a partial config gets its flat defaults materialized; no data loss). Not a bug.

So far config-persistence is solid on the merge path. Let me now check one asymmetry the prompt flagged that could be a genuine latent issue: `**`save_filter_config` catches corrupt config ? `raw_config={}`, silently dropping unknown keys**, but `load_policy_config` (which F2+ pipeline code will use to READ those keys) RAISES on the same corrupt config. This is an internal inconsistency. But more importantly ? is there a scenario where `save_filter_config` **silently drops** valid pipeline keys?`

The catch is only ``(json.JSONDecodeError, ValueError)``. ``_policy_config_json_to_dict` raises `ValueError` for non-object JSON, `JSONDecodeError` for malformed. Both are caught ? `{}``. So a corrupt policy that a user's ``/set`` touches would silently lose the pipeline keys (but they're already unreadable/corrupt). This is acceptable degradation, low severity.

Let me now verify there are no OTHER callers or tests catching pydantic ``ValidationError`` from `load_filter_config` across the whole test suite and source.

34. user (2026-07-06T20:54:54.715Z)

```
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:259:      with
pytest.raises(sqlite3.IntegrityError):
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:367:      with
pytest.raises(sqlite3.IntegrityError):
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:527:      """A config with min_duration >
max_duration must raise ValidationError."""
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:528:      from pydantic import ValidationError
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:532:      with pytest.raises(ValidationError,
match="min_duration_s.*must be <="):
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:603:      from pydantic import ValidationError
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:607:      with pytest.raises(ValidationError):
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:631:      with
pytest.raises(sqlite3.IntegrityError, match="FOREIGN KEY"):
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:888:      with
pytest.raises(sqlite3.IntegrityError, match="UNIQUE"):
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:970:      with pytest.raises(ValueError,
match="source_id and provider_message_id"):
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:984:      with pytest.raises(ValueError,
match="provider='whatsapp'"):
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:1142:      with
pytest.raises(sqlite3.IntegrityError, match="UNIQUE"):
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:1517:      with pytest.raises(ValueError,
match="Cannot cast"):
```

35. user (2026-07-06T20:54:55.599Z)

```
C:\Users\avidu\Projects\wt-pr50\tests\test_command_service.py:91:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_command_service.py:121:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_command_service.py:139:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_command_service.py:157:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_command_service.py:183:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_bootstrap.py:38:        cfg = await
db.load_filter_config(conn, 123456)
C:\Users\avidu\Projects\wt-pr50\tests\test_bootstrap.py:61:        cfg = await
db.load_filter_config(conn, 42)
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:449:async def
test_load_filter_config_defaults_when_absent(conn: aiomysql.Connection) -> None:
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:451:    cfg = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:455:async def
test_save_and_load_filter_config_round_trip(conn: aiomysql.Connection) -> None:
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:461:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:469:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:492:    raw = await
db.load_policy_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:497:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:508:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:522:        loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:559:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:570:        loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:589:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:1176:    """save_filter_config ?
load_filter_config round-trips through policies."""
C:\Users\avidu\Projects\wt-pr50\tests\test_db.py:1180:    loaded = await
db.load_filter_config(conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:348:    cfg = await
db.load_filter_config(memory_conn, 1)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:358:    loaded = await
db.load_filter_config(memory_conn, 1)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:371:    loaded = await
db.load_filter_config(memory_conn, 1)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:406:    loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:541:    loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:555:    loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:572:    loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:608:    loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:620:    loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:636:    loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:647:    loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:699:    loaded = await
```



```

db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:713: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:745: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:756: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:775: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:787: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:807: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:907: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:919: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:938: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:950: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:965: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:1025: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:1041: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:1052: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:1063: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:1071: cfg = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:1158: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:1173: loaded = await
db.load_filter_config(memory_conn, USER_ID)
C:\Users\avidu\Projects\wt-pr50\tests\test_commands.py:1185: loaded = await
db.load_filter_config(memory_conn, USER_ID)

```

36. assistant (2026-07-06T20:55:04.976Z)

No test or caller narrows on `pydantic.ValidationError` from `load\_filter\_config`. The exception-type change has no live consumer. Low severity.

Now let me examine one genuine concern I want to confirm: the `**`save_policy_config`` created\_at/updated\_at handling and the ``existing.id`` reuse\*\*. When ``save_policy_config`` is called with an existing default policy, it reuses ``existing.id``. But ``upsert_policy`` conflicts on ``(account_id, name)``, not ``id`` ? so the ``id`` passed is actually ignored on the UPDATE path (INSERT provides id, but ON CONFLICT updates by name). Let me check: is there a scenario where a NON-default policy named "default" or an id collision causes trouble?

Actually a more subtle issue: ``save_policy_config`` always writes ``name="default", is_default=True``. If ``get_default_policy`` returns a policy whose ``name`` is NOT "default" (default policy could theoretically have a different name), then ``save_policy_config`` upserts a NEW row with ``name="default" is_default=True``, colliding with the partial unique index ``idx_policies_account_default``. Let me test this edge case.

37. user (2026-07-06T20:55:17.450Z)

Default policy name: my-custom-default
save_filter_config -> IntegrityError: UNIQUE constraint failed: policies.account_id

38. assistant (2026-07-06T20:55:24.959Z)

This is a real edge case: if the account's default policy is named something other than `"default"`, `save_policy_config` raises an `IntegrityError` and the pipeline key is lost/unsaveable. However ? in F1, the ONLY way a default policy gets created is migration 003 (name='default') or these shims (name='default'). So a non-"default" default policy cannot arise in the current codebase. This is a latent/defensive concern, not a live bug. Medium-low. I'll note it but rank it appropriately.

Let me confirm there's no other path that creates default policies with other names, and check `bootstrap.py` and any policy-creating code.

39. user (2026-07-06T20:55:26.067Z)

```
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\backfill.py:362:
name=f"backfill-{user.id}-{duration_label}",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\manager.py:79:
asyncio.create_task(client_runner(session), name=f"session-{session.account_id}")
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\policy.py:41:
stage_name=stage_name,
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\policy.py:50:
stage_name=stage_name,
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\models.py:192: Exactly one policy per
account carries ``is_default=True``; the legacy
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:619: # (Option 2A).
is_default=1 so the legacy load_filter_config shim reads
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:767: "SELECT 1 FROM
sqlite_master WHERE type='table' AND name=?", (table,)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:804:
display_name=row["display_name"],
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:885: name=row["name"],
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:945:
display_name=excluded.display_name"",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1245:async def upsert_policy(conn:
aiosqlite.Connection, policy: Policy) -> Policy:
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1248: Exactly one policy per
account should carry ``is_default=True``; callers
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1282: ""Return the account's
default policy (``is_default=1``).""
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1328: await upsert_policy(
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1333: name="default",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1335: is_default=True,
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1753:
``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1843: "upsert_policy",
```

40. assistant (2026-07-06T20:55:34.370Z)

Confirmed: no other code creates a default policy with a name other than "default". So the `IntegrityError` edge case is purely defensive/latent ? it can't be triggered by current code paths. It stays a low-severity note since `save_policy_config`'s hardcoded `name="default"` diverges from `get_default_policy`'s name-agnostic lookup.

I have thoroughly investigated the config-persistence dimension. The merge order is correct, round-trips are stable, unknown keys (flat and nested) survive. The only genuine findings are asymmetries/latent issues, none of which is a live data-loss bug in this PR's happy or realistic paths. Let me record findings.

The exception-type change: real behavior change (pydantic `ValidationError` ? `ValueError/JSONDecodeError`), but no caller catches the old type, so it's low. The save/load corrupt-config asymmetry is a real inconsistency but acceptable degradation. The non-"default" name `IntegrityError` is latent.

Let me report these honestly with appropriate (low) severities.

41. user (2026-07-06T20:56:00.618Z)

Structured output provided successfully